

Final Paper

Subject: Advanced Distributed Systems

멀티 에이전트 공유 메모리 시스템에서의
계층형 일관성 모델 및
충돌 해소 메커니즘 설계

위바오치

담당교수 : 백 건 효

June 2026

Major of Computer Science and Technology
Graduate School of Youngsan University

목 차

LIST OF TABLES	iii
LIST OF FIGURES	iv
제 1 장 서론	1
1.1 연구 배경	1
1.2 연구 동기 및 문제 제기	1
1.3 연구 목표 및 기여	2
1.4 연구 문제	2
1.5 논문의 구성	3
제 2 장 관련 연구	4
2.1 멀티 에이전트 메모리 구조 및 공유 메모리 연구	4
2.2 분산 시스템 일관성 모델 및 버전 관리 연구	5
2.3 충돌 탐지, 중재 전략 및 의미적 조정 연구	5
2.4 전체 연구동향 종합	6
2.5 기존 연구의 공통 한계	6
2.6 본 연구의 차별성	7
제 3 장 시스템 모델 및 설계	8
3.1 시스템 가정 및 범위	8
3.2 멀티 에이전트 공유 메모리 형식 모델	8
3.3 세 가지 충돌 유형 형식 정의	9
3.4 계층형 일관성 모델 설계	9
3.5 충돌 해소 메커니즘 설계	10
제 4 장 구현 및 실험 설계	12
4.1 프로토타입 구현 환경	12

4.2 전체 시스템 아키텍처.....	12
4.3 실험 설계.....	13
4.4 비교군 구성.....	13
4.5 평가 지표.....	14
4.6 실험 절차.....	14
제 5 장 결과 및 분석	16
5.1 실험 결과 개요.....	16
5.2 비교군별 종합 성능 결과.....	16
5.3 시나리오별 정확도 분석.....	18
5.4 소거 실험(Ablation Study) 분석	19
5.5 지연시간 분석.....	21
5.6 결과 해석 및 논의.....	22
제 6 장 결론 및 시사점	26
6.1 연구 요약.....	26
6.2 연구의 시사점.....	27
6.2.1 학술적 시사점.....	27
6.2.2 실무적 시사점.....	28
6.3 연구의 한계.....	28
6.4 향후 연구 방향.....	29
참고문헌.....	31
국문 초록.....	33
ABSTRACT	35
APPENDIX	37

LIST OF TABLES

표 1. 비교군별 종합 성능 결과.....	16
표 2. 시나리오별 정확도 및 LLM 호출 횟수 비교	18
표 3. 소거 실험(Ablation Study) Δ 분석 결과.....	19
표 4. 시나리오별 지연시간 상세 분석.....	21

LIST OF FIGURES

그림 1. 비교군별 종합 정확도 및 평균 지연 시간 비교	17
그림 2. 시나리오별 비교군 정확도 비교	19
그림 3. 소거 실험 전후 성능 변화	20
그림 4. 비교군별 지연시간 분포 (Box Plot, N=90)	22

제 1 장 서론

1.1 연구 배경

최근 대규모 언어 모델(LLM) 기반 에이전트 시스템은 단일 에이전트 중심 구조에서 다수의 에이전트가 역할을 분담하여 협업하는 멀티 에이전트 구조로 빠르게 확장되고 있다. 이러한 변화는 복잡한 작업을 계획, 실행, 검토 단계로 분리하여 처리할 수 있게 한다는 점에서 높은 활용 가능성을 보이지만, 동시에 에이전트 간 상태와 지식을 안정적으로 공유할 수 있는 메모리 체계의 중요성을 크게 부각시킨다. 특히 여러 에이전트가 동일한 작업 맥락을 유지하면서 중간 결과를 재사용하고 의사결정의 근거를 축적하기 위해서는 공유 메모리의 구조와 운영 방식이 시스템 성능 및 신뢰성의 핵심 요소가 된다. 따라서 멀티 에이전트 환경에서 공유 메모리를 어떻게 설계하고 일관되게 관리할 것인가는 최근 AI 시스템 연구에서 중요한 과제로 부상하고 있다.

1.2 연구 동기 및 문제 제기

그러나 멀티 에이전트 공유 메모리 환경에서는 여러 에이전트가 동일한 메모리 항목에 동시에 접근하고 갱신하는 과정에서 다양한 충돌이 발생할 수 있다. 예를 들어 특정 정보가 일부 에이전트에게만 보이거나 늦게 전파되는 가시성 충돌, 최신 버전보다 오래된 기록이 뒤늦게 반영되는 순서성 충돌, 그리고 서로 다른 에이전트가 동일한 주제에 대해 의미적으로 상반된 결론을 기록하는 의미적 충돌이 대표적이다. 이러한 충돌이 적절히 관리되지 않으면 메모리 상태의 불일치가 누적되어 작업 성공률이 저하되고, 최종 결과의 신뢰성 또한 크게 훼손될 수 있다. 더 나아가 작업 메모리와 장기 메모리는 접근 빈도와 지연 허용 범위가 다르므로, 모든 메모리에 동일한 일관성 수준을 일률적으로 적용하는 방식만으로는 실제 협업 환경의 요구를 충분히 만족시키기 어렵다.

기존 연구에서는 멀티 에이전트 메모리를 공유 메모리와 분산 메모리 관점에서 개념적으로 설명하거나, 역할 기반 에이전트를 활용한 적응형 메모리 관리 방안을 제시한 바 있다. 그러나 많은 선행연구는 충돌 유형의 형식적 정의, 구현 가능한 일관성 프로토콜, 복수의 충돌 해소

전략에 대한 체계적 비교, 그리고 언어 의미 수준에서 발생하는 충돌의 처리 방안까지는 충분히 다루지 못하였다. 한편 분산 시스템 분야에서 제안된 강한 일관성, 최종 일관성, 인과 일관성과 같은 모델은 중요한 이론적 기반을 제공하지만, 이를 LLM 기반 멀티 에이전트 메모리 환경에 직접 적용하기 위해서는 메모리 계층 구조와 의미적 판단 요소를 함께 고려할 필요가 있다. 즉, 기존 연구는 메모리 아키텍처의 개념적 논의와 실제 충돌 관리 메커니즘의 실증적 검증 사이에 여전히 연구 공백을 남기고 있다.

1.3 연구 목표 및 기여

이에 본 연구에서는 멀티 에이전트 공유 메모리 시스템을 대상으로 작업 메모리에는 강한 일관성을, 장기 메모리에는 최종 일관성을 적용하는 계층형 일관성 모델을 설계하고자 한다. 또한 타임스탬프 우선, 신뢰도 우선, 리더 에이전트 중재 전략과 함께 LLM 기반 Judge Agent 를 활용한 의미적 충돌 탐지 및 조정 메커니즘을 통합적으로 제안한다. 이를 위해 Python 기반 프로토타입과 SQLite 공유 메모리 저장소를 구현하고, Planner·Executor·Reviewer 의 세 역할 에이전트가 참여하는 실험 환경에서 동시 쓰기 충돌, 버전 순서 충돌, 의미적 충돌 시나리오를 반복 수행하여 성능을 평가하였다. 구체적으로 충돌 해결 정확도, 메모리 일관성 유지율, 작업 최종 성공률, 평균 충돌 해결 지연을 중심으로 제안 방식의 효과를 분석함으로써, 본 연구는 멀티 에이전트 공유 메모리 환경에서 실용적인 일관성 설계 방향과 충돌 해소 메커니즘의 구현 가능성을 제시한다.

1.4 연구 문제

RQ1. 멀티 에이전트 공유 메모리 환경에서 발생하는 가시성·순서성·의미적 충돌은 제안 방식에 의해 기존 방식보다 더 정확하게 해결되는가?

RQ2. 작업 메모리와 장기 메모리에 서로 다른 일관성 수준을 적용하는 계층형 일관성 모델은 단일 일관성 모델 대비 정확도와 지연 시간의 균형을 향상시키는가?

RQ3. 타임스탬프 우선, 신뢰도 우선, 리더 에이전트 중재, Judge Agent 기반 의미 판단 전략 중 어떤 방식이 충돌 유형별로 가장 효과적인가?

RQ4. LLM 기반 Judge Agent 는 구조적 버전 관리 방식으로 탐지하기 어려운 의미적 충돌을 효과적으로 탐지하고 조정할 수 있는가?

RQ5. 소거 실험(Ablation Study)을 통해 구조적 충돌 해소 모듈과 Judge Agent 모듈의 개별 기여도를 분리하여 확인할 수 있는가?

1.5 논문의 구성

본 논문의 구성은 다음과 같다. 제 2 장에서는 멀티 에이전트 메모리 구조, 분산 시스템 일관성 모델, 충돌 해소 전략에 관한 선행연구를 검토하고 본 연구의 차별성을 제시한다. 제 3 장에서는 시스템 모델, 충돌 유형의 형식적 정의, 계층형 일관성 모델 및 충돌 해소 메커니즘의 설계를 상세히 기술한다. 제 4 장에서는 프로토타입 구현과 실험 설계, 비교군 구성 및 평가 방법을 설명한다. 제 5 장에서는 8 개 비교군에 대한 실험 결과를 제시하고 소거 실험 및 지연시간 분석을 포함한 종합적 해석을 수행한다. 마지막으로 제 6 장에서는 연구의 결론, 학술적·실무적 시사점, 한계 및 향후 연구 방향을 제시한다.

제 2 장 관련 연구

본 장에서는 본 연구와 관련된 선행연구를 검토하고, 기존 연구의 한계를 바탕으로 본 연구의 차별성을 제시한다.

대규모 언어 모델(LLM) 기반 시스템이 단일 에이전트 중심 구조를 넘어 다수의 에이전트가 역할을 분담하여 협업하는 형태로 확장되면서, 에이전트 간 정보 공유와 상태 동기화를 지원하는 메모리 구조의 중요성이 크게 증가하고 있다. 특히 멀티 에이전트 환경에서는 작업 계획, 실행 결과, 중간 추론 근거, 평가 피드백과 같은 정보가 여러 에이전트에 의해 반복적으로 읽히고 갱신되므로, 공유 메모리의 구조와 운용 방식이 전체 시스템의 신뢰성 및 성능을 좌우한다. 이러한 배경에서 기존 선행연구는 대체로 멀티 에이전트 메모리 구조 연구, 분산 시스템 일관성 모델 및 버전 관리 연구, 충돌 탐지 및 의미적 조정 연구의 세 흐름으로 구분할 수 있다.

2.1 멀티 에이전트 메모리 구조 및 공유 메모리 연구

첫째, 멀티 에이전트 메모리 구조 연구는 에이전트 간 협업을 지원하기 위한 메모리의 저장 형태, 접근 방식, 계층 구조를 중심으로 발전해 왔다. 기존 연구에서는 멀티 에이전트 메모리를 공유 메모리와 분산 메모리의 관점에서 구분하고, 에이전트가 공통 상태를 활용하여 협업 효율을 높일 수 있는 구조를 제시하였다. 또한 일부 연구는 작업 메모리, 장기 메모리, 외부 도구 기억을 구분하는 계층형 구조나, 역할 기반 에이전트가 필요에 따라 메모리를 검색·갱신하는 적응형 메커니즘을 제안함으로써 에이전트 협업의 확장 가능성을 보여주었다.

그러나 이러한 연구의 상당수는 메모리 구조의 개념적 유용성을 설명하는 데 초점을 두고 있으며, 실제 멀티 에이전트 환경에서 발생하는 동시 접근, 기록 순서 역전, 버전 충돌, 정보 가시성 문제를 형식적으로 정의하고 검증하는 수준에는 이르지 못하였다. 특히 공유 메모리에 대한 읽기·쓰기 규칙과 충돌 관리 정책이 구체적으로 설계되지 않은 경우가 많아, 실제 구현 단계에서 시스템 일관성을 어떻게 유지할 것인지에 대한 논의가 상대적으로 부족하다. 따라서 기존의 메모리 구조 연구는 협업을 위한 아키텍처 방향을 제시하였다는 의의가 있으나, 일관성 유지와 충돌 해소를 포함한 실증적 검토에는 한계를 가진다.

2.2 분산 시스템 일관성 모델 및 버전 관리 연구

둘째, 분산 시스템 일관성 모델 연구는 여러 노드가 공유 상태를 유지하는 환경에서 데이터의 정합성을 보장하기 위한 이론적 기반을 제공해 왔다. 강한 일관성(Linearizability) [3]은 모든 연산이 단일한 전역 순서를 따르는 것처럼 보장함으로써 높은 정확성을 제공하지만, 동시성 제어 비용과 지연 증가를 수반할 수 있다. 반면 최종 일관성(Eventual Consistency) [4]은 즉각적인 정합성보다 가용성과 확장성을 우선시하며, 시간이 경과하면 동일한 상태로 수렴하도록 설계된다. 인과 일관성(Causal Consistency) [5]은 연산 사이의 인과 관계를 유지하면서 강한 일관성과 최종 일관성 사이의 절충점을 제공한다. 이와 함께 버전 번호, 타임스탬프 [7], 복제 로그, 합의 기반 갱신 규칙 등은 분산 환경에서 순서성 충돌과 갱신 경쟁을 완화 [8]하기 위한 대표적 기법으로 활용되어 왔다.

그러나 기존 분산 시스템의 일관성 모델은 주로 키-값 데이터나 트랜잭션 중심의 구조적 정합성을 전제로 발전해 왔기 때문에, 자연어 텍스트와 추론 결과를 포함하는 LLM 기반 멀티 에이전트 메모리 환경에 그대로 적용하기에는 한계가 있다. 예를 들어 작업 메모리와 장기 메모리는 접근 빈도, 지연 허용 범위, 갱신 목적이 서로 다르므로 동일한 일관성 수준을 일률적으로 적용할 경우 정확도와 성능 사이의 균형을 확보하기 어렵다. 또한 텍스트 기반 메모리는 동일한 키에 저장된 내용이 구조적으로는 충돌하지 않더라도 의미적으로 상반될 수 있으므로, 전통적인 버전 관리만으로는 충분하지 않다. 따라서 기존 일관성 모델은 중요한 이론적 토대를 제공하지만, 멀티 에이전트 공유 메모리 환경에 적합한 계층형 일관성 설계로 확장될 필요가 있다.

2.3 충돌 탐지, 중재 전략 및 의미적 조정 연구

셋째, 충돌 탐지 및 조정 연구는 다수의 주체가 동일한 데이터 또는 메모리 항목을 갱신할 때 발생하는 충돌을 탐지하고 해결하기 위한 전략을 다루고 있다. 기존 연구와 실무 시스템에서는 일반적으로 타임스탬프 우선 방식, 최신 버전 우선 방식, 리더 또는 중앙 조정자 기반 중재 방식, 신뢰도 또는 우선순위 기반 선택 방식 등이 활용되어 왔다. 이러한 접근은 동시 쓰기 충돌이나

순서성 충돌과 같은 구조적 문제를 비교적 단순한 규칙으로 처리할 수 있다는 장점을 가진다. 최근에는 LLM 을 활용하여 상충되는 두 텍스트 기록의 의미적 모순 여부를 판별하거나, 다수의 응답 중 더 타당한 내용을 선택하도록 하는 Judge Agent 또는 평가자 에이전트 방식도 점차 논의되고 있다.

그러나 기존 충돌 해소 연구는 대체로 구조적 충돌의 탐지와 해결에 초점을 두고 있어, 동일한 주제에 대한 상반된 결론이나 문맥적 모순과 같은 의미적 충돌을 체계적으로 다루지 못한 경우가 많다. 또한 타임스탬프 우선, 신뢰도 우선, 리더 중재와 같은 다양한 전략을 동일한 실험 조건에서 비교하고, 정확도와 지연 시간의 균형을 함께 평가한 연구도 상대적으로 제한적이다. LLM 기반 의미 판단 접근 역시 가능성을 보여주고 있으나, 실제 멀티 에이전트 공유 메모리 시스템에서 구조적 충돌 처리와 통합하여 설계한 연구는 아직 충분하지 않다. 따라서 구조적 충돌과 의미적 충돌을 연계하여 탐지·조정하는 통합 메커니즘에 대한 연구가 필요하다.

2.4 전체 연구동향 종합

이상의 선행연구를 종합하면, 관련 연구는 크게 세 가지 방향으로 발전해 왔다고 정리할 수 있다. 첫째, 멀티 에이전트 협업을 지원하기 위한 메모리 구조와 검색 메커니즘의 설계가 활발히 논의되면서, 공유 메모리의 필요성과 계층적 설계 가능성이 제시되었다. 둘째, 분산 시스템의 일관성 이론과 버전 관리 기법은 공유 상태의 정합성을 설명하는 핵심 토대로 기능하며, 멀티 에이전트 메모리에도 적용 가능한 원리를 제공하였다. 셋째, 충돌 탐지와 중재 전략 연구는 구조적 충돌을 해결하기 위한 다양한 규칙 기반 접근 [9]을 축적해 왔고, 최근에는 의미적 충돌까지 다루기 위한 LLM 기반 판단 메커니즘이 추가되고 있다.

2.5 기존 연구의 공통 한계

기존 연구의 공통 한계는 다음과 같이 정리될 수 있다. 첫째, 많은 연구가 멀티 에이전트 메모리의 구조와 활용 가능성을 개념적으로 제시하였으나, 공유 메모리 접근 규칙, 버전 관리 방식, 충돌 유형의 형식적 정의를 포함한 구현 수준의 프로토콜은 충분히 제시하지 못하였다. 둘째, 분산 시스템의 전통적 일관성 모델은 중요한 이론적 기반을 제공하지만, 작업 메모리와 장기 메모리의 특성 차이, 자연어 기반 메모리의 의미적 모순 가능성, 에이전트 역할 차이와 같은 요소를 동시에

반영하지 못하였다. 셋째, 기존 충돌 해소 연구는 주로 구조적 충돌 해결에 집중하여, 텍스트 내용 간 상충이나 판단 결과의 모순과 같은 의미적 충돌을 실질적으로 다루지 못한 경우가 많다. 넷째, 타임스탬프 우선, 신뢰도 우선, 리더 에이전트 중재, 의미적 Judge Agent 등 다양한 전략의 성능을 동일 조건에서 비교하여 정확도, 일관성 유지율, 작업 성공률, 지연 시간을 종합적으로 평가한 연구는 상대적으로 부족하다.

2.6 본 연구의 차별성

따라서 본 연구는 멀티 에이전트 공유 메모리 시스템을 대상으로 작업 메모리와 장기 메모리를 구분하고, 각각에 적합한 일관성 수준을 적용하는 계층형 일관성 모델을 제안한다. 구체적으로 작업 메모리에는 강한 일관성을 적용하여 현재 작업 맥락의 정합성을 우선 확보하고, 장기 메모리에는 최종 일관성을 적용하여 확장성과 유연성을 확보하고자 한다. 또한 본 연구는 가시성 충돌, 순서성 충돌, 의미적 충돌의 세 유형을 형식적으로 정의하고, 타임스탬프 우선, 신뢰도 우선, 리더 에이전트 중재, LLM 기반 Judge Agent 를 포함하는 충돌 해소 메커니즘을 통합적으로 설계한다. 아울러 Python 기반 프로토타입과 SQLite 공유 메모리 저장소를 구현하고, Planner·Executor·Reviewer 의 세 역할 에이전트가 참여하는 실험 환경에서 동시 쓰기 충돌, 버전 순서 충돌, 의미적 충돌 시나리오를 반복 수행함으로써 제안 방식의 효과를 검증하였다. 이 과정에서 충돌 해결 정확도, 메모리 일관성 유지율, 작업 최종 성공률, 평균 충돌 해결 지연을 핵심 지표로 활용하여 기존 방식과의 차이를 비교하였다. 이러한 점에서 본 연구는 메모리 구조, 일관성 수준, 충돌 중재 전략, 의미적 조정을 하나의 프레임워크로 통합하고, 실제 구현 및 비교 실험을 통해 실용적 설계 방향을 제시한다는 점에서 기존 연구와 차별성을 가진다.

제 3 장 시스템 모델 및 설계

이와 같은 선행연구 검토를 바탕으로, 본 장에서는 멀티 에이전트 공유 메모리 시스템의 형식 모델, 충돌 유형의 정의, 계층형 일관성 모델 및 충돌 해소 메커니즘의 설계를 상세히 기술한다.

3.1 시스템 가정 및 범위

본 연구는 설계 과학 연구(Design Science Research) 방법론을 채택하여, 멀티 에이전트 공유 메모리 시스템을 위한 인공물(Artifact)로서 계층형 일관성 모델과 충돌 해소 메커니즘을 설계하고 실험적으로 평가한다. 시스템은 다음과 같은 가정 하에 설계되었다.

에이전트 수는 3 개(Planner, Executor, Reviewer)로 고정되며, 각 에이전트는 특정 역할을 수행한다.

공유 메모리는 SQLite 기반 단일 머신 환경에서 운영되며, 모든 에이전트가 동일한 메모리 저장소에 접근 가능하다.

네트워크는 안정적이며, 메시지 손실이나 노드 장애는 발생하지 않는 것으로 가정한다.

메모리는 작업 메모리(Working Memory)와 장기 메모리(Long-term Memory)의 두 계층으로 구성된다.

3.2 멀티 에이전트 공유 메모리 형식 모델

공유 메모리 시스템은 메모리 항목(Entry)의 집합으로 정의된다. 각 메모리 항목은 키(key), 값(value), 버전 번호(version), 타임스탬프(timestamp), 작성자 에이전트 식별자(agent_id)로 구성된다. 메모리 항목은 키를 통해 고유하게 식별되며, 각 갱신 연산마다 버전 번호가 1 씩 증가한다. 읽기 연산은 키와 메모리 유형을 지정하여 현재 저장된 값을 반환하고, 쓰기 연산은 키, 값, 에이전트 식별자를 지정하여 새로운 메모리 항목을 생성하거나 기존 항목을 갱신한다.

메모리 항목에 대한 형식 정의는 다음과 같다: $\text{MemoryEntry} = \langle \text{key}, \text{value}, \text{version}, \text{timestamp}, \text{agent_id}, \text{memory_type} \rangle$. 여기서 $\text{version} \in \mathbb{N}$ 은 갱신 횟수를, $\text{timestamp} \in \mathbb{R}^+$ 는 Unix 시간을, $\text{memory_type} \in \{\text{WORKING}, \text{LONG_TERM}\}$ 은 메모리 계층을 나타낸다.

3.3 세 가지 충돌 유형 형식 정의

본 연구는 멀티 에이전트 공유 메모리 환경에서 발생하는 충돌을 다음과 같이 세 가지 유형으로 형식적으로 정의한다.

가시성 충돌 (Visibility Conflict): Agent A 가 메모리 키 k 에 대해 쓰기를 수행한 후, Agent B 가 k 를 읽을 때 A 의 갱신이 반영되지 않은 상태를 읽는 경우 발생한다. 이는 에이전트가 최신 버전을 인지하지 못한 상태(stale read)를 의미하며, 에이전트의 $\text{last_seen_version}[k] < \text{current_version}[k]$ 인 조건으로 탐지된다.

순서성 충돌 (Ordering Conflict): Agent B 가 과거 버전을 기반으로 쓰기를 수행하여, Agent A 가 이미 갱신한 최신 값을 덮어쓰는 경우 발생한다. 이는 $0 < \text{agent.last_seen_version}[k] < \text{current_version}[k]$ 인 조건에서 탐지되며, 가시성 충돌이 쓰기 동작으로 이어진 특수한 경우이다.

의미적 충돌 (Semantic Conflict): 두 에이전트가 동일한 키에 대해 구조적으로는 정상적인 쓰기를 수행하였으나, 기록된 내용이 의미적으로 모순되거나 상호 배타적인 결론을 포함하는 경우 발생한다. 이는 버전 비교만으로는 탐지할 수 없으며, 자연어 의미 분석이 필요한 충돌 유형이다. 예를 들어, Agent A 가 "시스템이 운영 배포 가능 상태"라고 기록하고 Agent B 가 동일 키에 "시스템에 심각한 버그가 있어 배포 불가"라고 기록하는 경우가 이에 해당한다.

3.4 계층형 일관성 모델 설계

본 연구는 작업 메모리와 장기 메모리에 서로 다른 일관성 수준을 적용하는 계층형 일관성 모델(Hierarchical Consistency Model)을 설계한다. 이러한 계층적 접근의 근거는 두 메모리 계층의 접근 특성과 일관성 요구 수준이 근본적으로 다르기 때문이다.

작업 메모리 (Working Memory) — 강한 일관성: 작업 메모리는 현재 진행 중인 작업 계획(task_plan), 작업 상태(task_status), 현재 단계(current_step)와 같은 실시간 협업 정보를 저장한다. 접근 빈도가 높고, 잘못된 정보로 인한 연쇄적 오류 발생 위험이 크기 때문에, 본 연구는

작업 메모리에 잠금(lock) 기반의 강한 일관성을 적용한다. 강한 일관성 하에서는 모든 읽기/쓰기 연산이 직렬화되어 실행되며, 에이전트는 항상 최신 값을 읽을 수 있다.

장기 메모리 (Long-term Memory) — 최종 일관성: 장기 메모리는 지식 베이스(knowledge_base), 의사결정 로그(decision_log), 피드백 이력(feedback_history)과 같은 축적형 정보를 저장한다. 갱신 빈도가 상대적으로 낮고, 약간의 지연이 허용되므로, 본 연구는 장기 메모리에 최종 일관성을 적용한다. 최종 일관성 하에서는 쓰기 연산이 비차단(non-blocking) 방식으로 수행되며, 충돌 발생 시 last-write-wins 정책으로 수렴된다.

계층형 일관성 모델은 HierarchicalConsistency 클래스로 구현되며, 메모리 유형에 따라 StrongConsistency 또는 EventualConsistency 로 읽기/쓰기 연산을 위임한다. 또한 작업 메모리에서 장기 메모리로의 승급(upgrade_to_long_term) 메커니즘을 제공하여, 검증된 작업 메모리 항목을 장기 메모리로 이관할 수 있다.

3.5 충돌 해소 메커니즘 설계

본 연구는 구조적 충돌과 의미적 충돌을 통합적으로 처리하는 3 단계 충돌 해소 메커니즘을 설계한다.

1 단계: 충돌 탐지 (Conflict Detection). ConflictDetector 는 에이전트의 상태(AgentState)와 메모리 저장소의 현재 버전을 비교하여 가시성 충돌과 순서성 충돌을 탐지한다. 가시성 충돌은 $last_seen_version < current_version$ 일 때, 순서성 충돌은 $0 < last_seen_version < current_version$ 일 때 탐지된다.

2 단계: 구조적 사전 필터링 (Structural Pre-filtering). 탐지된 충돌에 대해 타임스탬프 우선, 신뢰도 우선, 리더 에이전트 중재의 세 가지 구조적 해소 전략 중 하나를 적용한다. 타임스탬프 우선 전략은 가장 최근에 기록된 값을 선택하고, 신뢰도 우선 전략은 더 높은 Trust Score(성공 횟수/총 시도 횟수)를 가진 에이전트의 값을 선택하며, 리더 에이전트 중재 전략은 Reviewer 에이전트의 규칙 기반 판단에 따라 값을 선택한다.

3 단계: Judge Agent 의미적 판단 (Semantic Judgment). 구조적 필터링으로 해결되지 않은 충돌, 특히 자연어 텍스트 간의 의미적 충돌에 대해서는 LLM 기반 Judge Agent 가 개입한다. Judge Agent 는 DeepSeek-chat [10] API [10]를 호출하여 두 값의 의미적 충돌 여부를 판별하고, 충돌 시 더 적절한 값을 선택한다. Judge Agent 의 프롬프트는 메모리 키, 에이전트 정보, 두 개의 충돌 값을 포함하며, JSON 형식으로 충돌 여부(conflict), 해결 방향(resolution), 판단 근거(reasoning)를 반환한다.

이러한 3 단계 메커니즘은 ConflictResolver 에 의해 통합 운영되며, ResolverConfig 를 통해 각 구성 요소의 활성화 여부를 조정할 수 있다. 사전 필터링이 적용된 경우, 자연어 수준의 의미 판단이 필요한 값에만 Judge Agent 를 선별적으로 호출함으로써 불필요한 LLM API 호출을 최소화한다.

제 4 장 구현 및 실험 설계

본 장에서는 제 3 장에서 설계한 시스템의 프로토타입 구현과 실험 설계를 기술한다.

4.1 프로토타입 구현 환경

프로토타입은 Python 3.12 기반으로 구현되었으며, 주요 구성 요소는 다음과 같다. 공유 메모리 저장소로는 SQLite 를 사용하였고, WAL(Write-Ahead Logging) [12] 모드를 활성화하여 동시 읽기/쓰기 성능을 확보하였다. LLM API 호출에는 DeepSeek-chat [10] 모델을 사용하였으며, OpenAI SDK [11]를 통해 DeepSeek 의 OpenAI 호환 엔드포인트에 접근하였다. 실험 데이터의 수집 및 통계 분석에는 NumPy 를 활용하였다.

프로토타입의 전체 소스 코드와 실험 데이터는 GitHub 저장소(<https://github.com/ybq9430/multi-agent-shared-memory-experiment>)에 공개되어 있으며, config.py 에서 API 키를 환경 변수로 설정한 후 재현 가능하다.

4.2 전체 시스템 아키텍처

시스템은 6 개의 핵심 모듈로 구성된다:

agents — BaseAgent(읽기/쓰기/신뢰도 추적), PlannerAgent, ExecutorAgent, ReviewerAgent, JudgeAgent(LLM 기반 의미적 판단)

memory — SharedMemoryStore(SQLite 기반 버전 관리 저장소), MemoryEntry, ConflictRecord, AgentState 데이터 모델

conflict — ConflictDetector(구조적 충돌 탐지), ConflictResolver(3 단계 해소 파이프라인 통합)

consistency — StrongConsistency(잠금 기반), EventualConsistency(비차단), HierarchicalConsistency(계층형 통합)

strategies — 타임스탬프 우선(resolve_timestamp), 신뢰도 우선(resolve_trust_score), 리더 중재(resolve_leader)

experiments — ExperimentRunner(실행 루프), ScenarioGenerator(3 종 시나리오), build_groups(8 개 비교군), MetricsCollector, 결과 출력 및 CSV 내보내기

데이터 흐름은 ScenarioGenerator → Agent writes → ConflictDetector → ConflictResolver(Judge Agent 포함) → MetricsCollector → Reporter 순으로 진행된다.

4.3 실험 설계

실험의 목적은 멀티 에이전트 공유 메모리 환경에서 제안하는 계층형 일관성 모델과 충돌 해소 메커니즘이 기존 방식 대비 어떠한 장점과 한계를 가지는지를 검증하는 데 있다. 본 실험은 10 주차에 수립한 실험계획서에 따라 수행되었다.

실험 환경은 다음과 같이 구성되었다: Python 3.12, DeepSeek-chat [10] API [10], SQLite(WAL 모드), Windows 11 환경에서 3 개 에이전트(Planner, Executor, Reviewer)가 참여하였다. 실험은 8 개 비교군에 대해 3 개 충돌 시나리오를 각 30 회 반복 수행하여 총 720 회 실행되었다. 난수 시드는 42 로 고정하여 비교군 간 동일한 워크로드 분포를 보장하였다.

4.4 비교군 구성

비교군은 기존 방식과 제안 방식의 성능 차이를 확인하는 동시에, 소거 실험(Ablation Study)을 통해 핵심 모듈의 개별 기여도를 확인할 수 있도록 8 개로 구성하였다.

Group 1 (Baseline): 충돌 탐지 없음, 전략 없음, Judge Agent 없음. Last-write-wins 만 적용.

Group 2 (StrongOnly): 충돌 탐지 적용, 강한 일관성만 적용, 전략 및 Judge Agent 없음.

Group 3 (TimestampPriority): 충돌 탐지 + 타임스탬프 우선 전략 + 사전 필터링.

Group 4 (TrustScorePriority): 충돌 탐지 + 신뢰도 우선 전략 + 사전 필터링.

Group 5 (LeaderMediation): 충돌 탐지 + 리더 에이전트 중재 + 사전 필터링.

Group 6 (Ablation1_NoJudge): 계층형 일관성 + 모든 구조적 전략 + 사전 필터링, Judge Agent 제외.

Group 7 (Ablation2_NoPreFilter): 계층형 일관성 + 모든 구조적 전략 + Judge Agent 포함, 사전 필터링 제외.

Group 8 (FullProposed): 계층형 일관성 + 모든 구조적 전략 + Judge Agent + 사전 필터링. 제안 방식.

4.5 평가 지표

실험의 평가 지표는 다음과 같이 네 가지로 설정하였다:

충돌 해결 정확도 (Accuracy): 전체 반복 중 해결된 값이 정답(Ground Truth)과 일치한 비율(%)

메모리 일관성 유지율 (Consistency Rate): 전체 반복 중 충돌이 적극적으로 해소된 비율(%). Last-write-wins 로 방치되지 않은 비율.

작업 최종 성공률 (Task Success Rate): 해결된 값이 유효한 값으로 존재하는 비율(%)

평균 충돌 해결 지연 (Average Latency): 충돌 감지부터 해결까지 소요된 평균 시간(ms)

4.6 실험 절차

실험은 다음과 같은 절차로 수행되었다.

1. 프로토타입 초기화: SQLite 공유 메모리 테이블, 에이전트 정보, 버전 번호, 타임스탬프 필드를 초기화한다.

2. 워크로드 생성: ScenarioGenerator 를 통해 동시 쓰기 충돌, 버전 순서 충돌, 의미적 충돌에 해당하는 입력 데이터를 생성한다.

3. 비교군별 실행: 8 개 비교군 각각에 대해 3 개 시나리오 × 30 회 반복을 동일 조건에서 실행한다.

4. 충돌 탐지 기록: 각 실행에서 충돌 발생 여부, 충돌 유형, 관련 메모리 키, 에이전트 ID 를 기록한다.

5. 충돌 해소 수행: 비교군별 정책에 따라 최종 메모리 값을 결정하고 해결 결과를 저장한다.

6. Judge Agent 평가: 의미적 충돌 시나리오에서는 LLM 기반 Judge Agent 의 판정 결과와 정답 레이블을 비교한다.

7. 소거 실험 분석: Judge Agent 제거 조건(Ablation1)과 구조적 사전 필터링 제거 조건(Ablation2)을 제안 방식(FullProposed)과 비교하여 모듈별 Δ 정확도, Δ 지연, LLM 호출 횟수를 산출한다.

8. 평가 지표 산출: 충돌 해결 정확도, 메모리 일관성 유지율, 작업 성공률, 평균 지연 시간을 계산한다.

본 연구는 데이터 수집 및 활용 과정에서 개인정보 보호와 연구윤리 기준을 준수하였다. 실험 데이터는 시뮬레이션 환경에서 생성된 합성 데이터로서 실제 사용자 정보를 포함하지 않으며, 데이터 처리 절차는 연구윤리 체크리스트에 따라 점검되었다.

제 5 장 결과 및 분석

본 장에서는 제 4 장의 실험 설계에 따라 수행된 8 개 비교군의 실험 결과를 제시하고, 연구가설과 연결하여 해석한다.

5.1 실험 결과 개요

본 연구에서는 멀티 에이전트 공유 메모리 시스템에서 제안하는 계층형 일관성 모델과 충돌 해소 메커니즘의 유효성을 검증하기 위해, Python 기반 프로토타입과 SQLite 공유 메모리 저장소를 구현하고, 3 개 에이전트(Planner, Executor, Reviewer)가 참여하는 실험 환경에서 8 개 비교군에 대해 3 개 충돌 시나리오를 각 30 회 반복 수행하여 총 720 회의 실험을 실행하였다. 제안 방식(FullProposed)은 전체 시나리오 평균 93.3%의 정확도를 기록하여 Baseline(71.1%) 대비 22.2%p 향상된 성능을 보였다.

5.2 비교군별 종합 성능 결과

표 1 은 8 개 비교군의 세 시나리오에 대한 정확도, 메모리 일관성 유지율 및 평균 지연 시간을 종합한 결과이다. 충돌 탐지 및 해소 메커니즘을 적용한 모든 그룹은 일관성 유지율 100%를 달성한 반면, Baseline 과 StrongOnly 는 0%에 그쳤다. 종합 정확도 측면에서 제안 방식(FullProposed)이 93.3%로 가장 높았으며, 이는 Baseline(71.1%) 대비 22.2%p 향상된 수치이다.

표 1. 비교군별 종합 성능 결과

Group	Accuracy (%)	Consistency (%)	Success (%)	Avg Latency (ms)
1_Baseline	71.11	0.00	100.00	4.38
2_StrongOnly	71.11	0.00	100.00	4.92
3_TimestampPriority	91.11	100.00	100.00	4.35
4_TrustScorePriority	62.22	100.00	100.00	5.47
5_LeaderMediation	62.22	100.00	100.00	5.31
6_Ablation1_NoJudge	91.11	100.00	100.00	5.45
7_Ablation2_NoPreFilter	75.56	100.00	100.00	1485.75
8_FullProposed	93.33	100.00	100.00	478.72

표 1 에서 확인할 수 있듯이, 충돌 탐지기를 적용하지 않은 Baseline 과 강한 일관성만 적용한 StrongOnly 는 충돌이 발생해도 이를 탐지·해소하지 못하여 일관성 유지율 0%를 기록한 반면, 충돌 탐지기를 적용한 모든 비교군(Groups 3~8)은 일관성 유지율 100%를 달성하였다. 이는 충돌 해소 전략의 종류와 무관하게, 충돌 탐지 모듈 자체가 메모리 일관성 확보의 필요조건임을 시사한다.

한편 Judge Agent 를 포함하지 않은 Ablation2(NoPreFilter)는 종합 정확도 75.56%를 기록하여, 구조적 전략만으로는 의미적 충돌을 적절히 해결하지 못해 종합 성능이 하락함을 보여준다. 이는 사전 필터링이 결여된 상태에서 Judge Agent 가 구조적 충돌까지 처리하려다 오히려 오판하여 동시 쓰기 정확도가 46.67%로 하락한 데 기인한다.

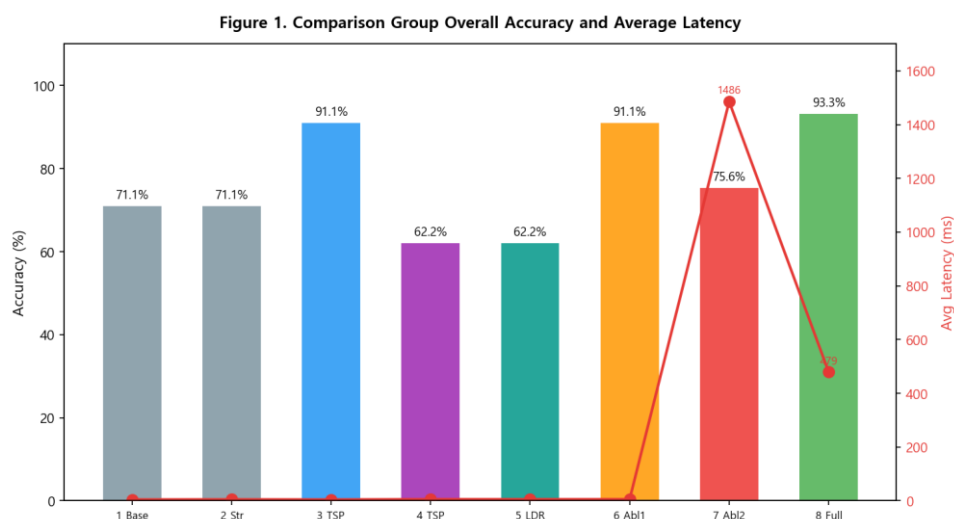


그림 1. 비교군별 종합 정확도 및 평균 지연 시간 비교

그림 1 은 8 개 비교군의 종합 정확도(막대)와 평균 지연 시간(선)을 함께 나타낸 이중축 그래프이다. FullProposed 는 93.3%의 가장 높은 종합 정확도를 달성하면서도 평균 지연 시간 478.7ms 로, Ablation2(1485.8ms) 대비 3.1 배 낮은 지연 시간을 기록하였다. 그룹 1~6 은 모두 10ms 이내의 낮은 지연 시간을 보였고, 특히 구조적 전략을 적용한 그룹들은 높은 정확도와 낮은 지연 시간을 동시에 달성하여 실용적 가치를 입증하였다.

5.3 시나리오별 정확도 분석

표 2 는 각 비교군의 시나리오별 정확도와 LLM 호출 횟수를 구분하여 보여준다. 동시 쓰기 충돌 시나리오에서는 TimestampPriority, Ablation1, FullProposed 가 100%의 정확도를 달성하였다. 순서 충돌 시나리오에서는 모든 비교군이 100%를 기록하였다. 의미적 충돌 시나리오에서는 Judge Agent 를 포함한 그룹(7, 8)이 80.0%로 가장 높은 정확도를 보였다.

표 2. 시나리오별 정확도 및 LLM 호출 횟수 비교

Group	Simul.Write (%)	Ordering (%)	Semantic (%)	LLM Calls
1_Baseline	40.00	100.00	73.33	0
2_StrongOnly	40.00	100.00	73.33	0
3_TimestampPriority	100.00	100.00	73.33	0
4_TrustScorePriority	60.00	100.00	26.67	0
5_LeaderMediation	60.00	100.00	26.67	0
6_Ablation1_NoJudge	100.00	100.00	73.33	0
7_Ablation2_NoPreFilter	46.67	100.00	80.00	90
8_FullProposed	100.00	100.00	80.00	30

FullProposed 는 30 회의 LLM 호출로 80.0%의 의미적 충돌 정확도를 달성한 반면, Ablation2 는 90 회의 LLM 호출에도 동시 쓰기 정확도가 46.7%에 그쳤다. 이는 사전 필터링 없이 Judge Agent 에 전적으로 의존할 경우, Judge Agent 가 버전 문자열과 같은 구조적 정보의 의미를 적절히 해석하지 못하여 구조적 충돌의 해결 정확도가 크게 저하됨을 의미한다. 또한 신뢰도 우선 및 리더 중재 전략은 의미적 충돌에서 26.7%로 하락하여, 단일 규칙 기반 접근의 한계를 확인할 수 있다.

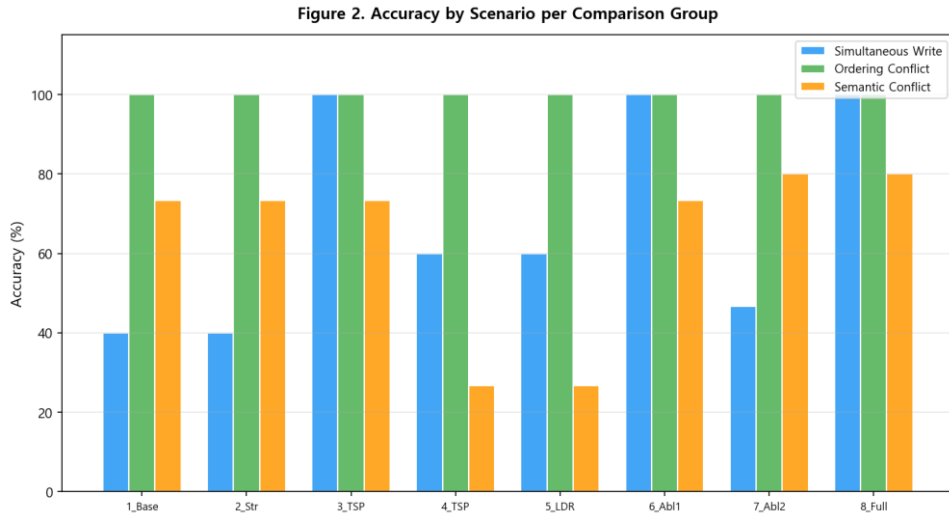


그림 2. 시나리오별 비교군 정확도 비교

그림 2 는 3 개 시나리오에 대한 8 개 비교군의 정확도를 그룹 막대그래프로 나타낸 것이다. 동시 쓰기와 순서 충돌 시나리오에서는 TimestampPriority, Ablation1, FullProposed 가 100%를 달성한 반면, 의미적 충돌에서는 Judge Agent 를 포함한 그룹만이 80.0%의 높은 정확도를 보였다.

5.4 소거 실험(Ablation Study) 분석

표 3 은 소거 실험을 통해 Judge Agent 모듈과 구조적 사전 필터링 모듈의 개별 기여도를 정량적으로 분석한 결과이다.

표 3. 소거 실험(Ablation Study) Δ 분석 결과

Comparison	Scenario	Δ Accuracy (%p)	Δ LLM Calls	Key Finding
Ablation1 (NoJudge)	semantic	-6.67	0	Judge Agent contributes to semantic accuracy
vs FullProposed	simult_write	0.00	0	Judge Agent not needed for structural conflicts
Ablation2 (NoPreFilter)	simult_write	-53.33	+60	Pre-filter critical for structural accuracy
vs FullProposed	semantic	0.00	+60	Unnecessary LLM calls without pre-filtering
Ablation2 (NoPreFilter)	ordering	0.00	+30	LLM overhead even for trivial cases

Ablation1(NoJudge)은 의미적 충돌에서 FullProposed 대비 6.67%p 낮은 정확도를 기록하여 Judge Agent 의 기여도를 확인할 수 있다. Ablation2(NoPreFilter)는 LLM 호출이 90 회로 FullProposed(30 회) 대비 60 회 증가(+200%)하였으며, 동시 쓰기 정확도가 46.7%로 FullProposed(100.0%) 대비 53.3%p 하락하였다. 이는 LLM 기반 Judge Agent 가 자연어 의미 충돌에 효과적이나 구조적 판별에는 한계가 있음을 의미하며, 사전 필터링이 없을 경우 Judge Agent 가 구조적 충돌까지 처리하려다 오히려 오판할 가능성이 증가함을 시사한다.

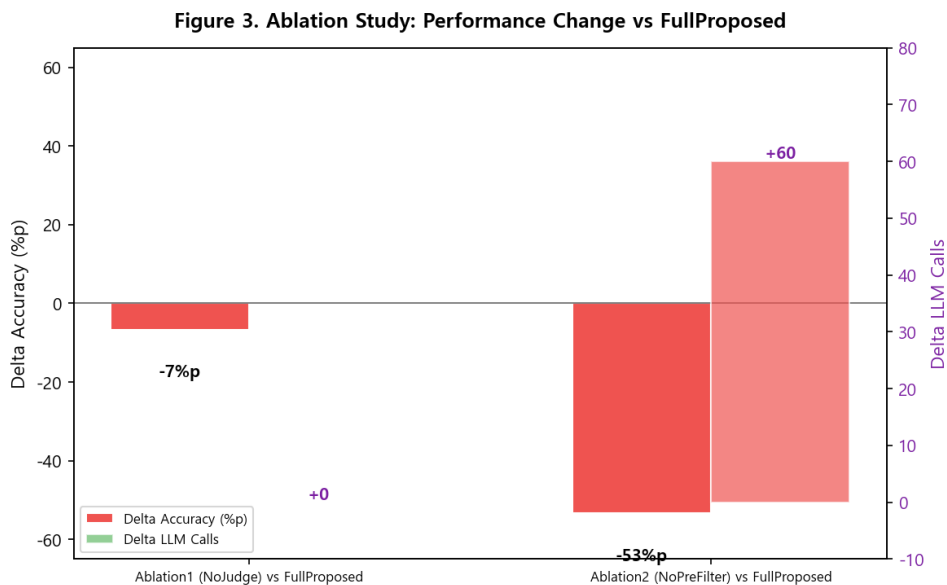


그림 3. 소거 실험 전후 성능 변화

그림 3 은 FullProposed 대비 Ablation1 과 Ablation2 의 핵심 지표 변화를 비교한 것이다. Judge Agent 제거 시 의미적 충돌 정확도가 하락하고, 사전 필터링 제거 시 LLM 호출이 급증하고 구조적 충돌 정확도가 크게 하락하여 두 모듈이 상호 보완적이며 제안 방식이 최적의 균형점임을 시각적으로 입증한다.

5.5 지연시간 분석

표 4 는 각 비교군의 시나리오별 평균 지연시간, 최소값, 최대값을 제시하여 지연시간의 변동성을 종합적으로 보여준다.

표 4. 시나리오별 지연시간 상세 분석

Group	Simul.Write (ms)	Ordering (ms)	Semantic (ms)	Overall Avg (ms)	Min (ms)	Max (ms)
1_Baseline	4.15	2.94	6.04	4.38	2.94	6.04
2_StrongOnly	5.37	3.93	5.44	4.92	3.93	5.44
3_TimestampPriority	5.31	2.65	5.11	4.35	2.65	5.31
4_TrustScorePriority	6.66	4.37	5.38	5.47	4.37	6.66
5_LeaderMediation	5.94	4.35	5.65	5.31	4.35	5.94
6_Ablation1_NoJudge	6.62	3.98	5.74	5.45	3.98	6.62
7_Ablation2_NoPreFilter	1647.83	1365.03	1444.40	1485.75	1365.03	1647.83
8_FullProposed	5.69	3.98	1426.48	478.72	3.98	1426.48

표 4 에서 확인할 수 있듯이, LLM 을 호출하지 않는 그룹(1~6)의 지연시간은 모든 시나리오에서 수 ms 이내로 매우 낮다. 반면 Ablation2 는 모든 시나리오에서 1300~1700ms 의 높은 지연시간을 보이며, 이는 매 실행마다 LLM API 를 호출하기 때문이다. FullProposed 는 구조적 충돌 시나리오에서는 LLM 을 호출하지 않아 10ms 미만의 낮은 지연시간을 유지하고, 의미적 충돌에서만 LLM 을 호출하여 지연시간이 증가한다.

제안 방식(FullProposed)은 평균 지연시간 478.7ms 를 기록하여 Ablation2(1485.8ms) 대비 3.1 배 낮은 지연시간을 보였다. 이는 사전 필터링이 불필요한 LLM 호출을 효과적으로 차단하여 지연시간의 평균뿐만 아니라 변동성까지 효과적으로 제어함을 의미하며, 실시간 협업이 요구되는 멀티 에이전트 시스템에서 사전 필터링의 실용적 가치를 뒷받침한다.

아울러 LLM API 응답 시간의 자연적 변동성으로 인해, Judge Agent 를 포함한 그룹(7, 8)의 의미적 충돌 시나리오에서 지연시간이 크게 증가하였다. 이는 DeepSeek-chat [10] API [10]의 응답 시간 변동성에 기인한 것으로, 실제 서비스 환경에서는 네트워크 지연과 부하에 따라 변동성이 더욱 커질 수 있다. 따라서 LLM 호출 횟수를 최소화하는 사전 필터링의 중요성은 정확도 측면뿐만 아니라 시스템 예측 가능성(predictability) 측면에서도 중요한 설계 고려사항이다.

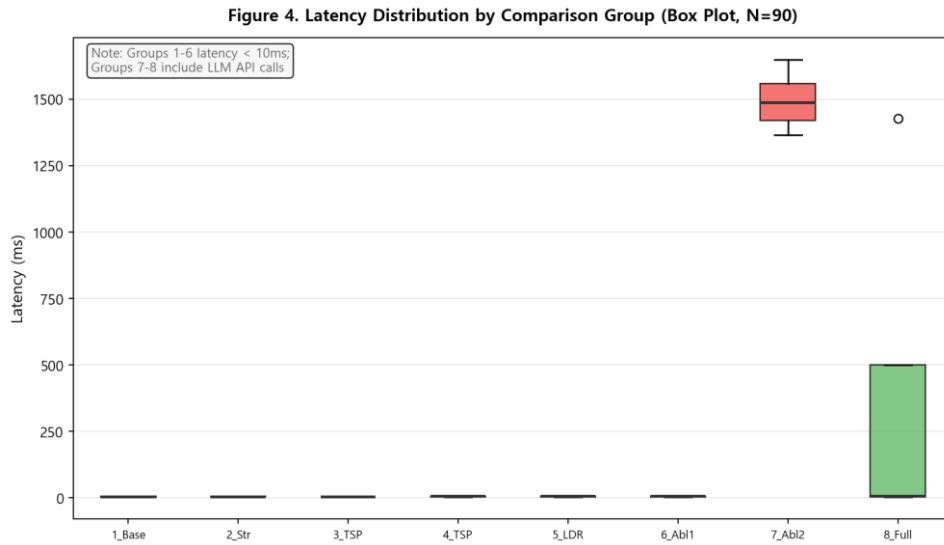


그림 4. 비교군별 지연시간 분포 (Box Plot, N=90)

그림 4는 8개 비교군의 전체 90회 실험(3개 시나리오 × 30회 반복)에 대한 지연시간 분포를 상자 그림(Box Plot)으로 나타낸 것이다. 그룹 1~6(저지연 그룹)의 지연시간은 모든 반복에서 10ms 이내로 매우 안정적인 반면, 그룹 7(Ablation2)은 세 시나리오 모두에서 1300~1700ms의 높고 넓은 분포를 보인다. 그룹 8(FullProposed)은 의미적 충돌 시나리오에서만 LLM 호출이 발생하여 분포가 넓어지나, 구조적 충돌에서는 저지연을 유지하여 그룹 7보다 전반적인 지연시간 분포가 현저히 낮다. 이는 구조적 사전 필터링이 LLM 호출 횟수를 줄여 지연시간의 평균뿐만 아니라 변동성(분산)까지 효과적으로 제어함을 의미한다.

5.6 결과 해석 및 논의

이상의 실험 결과를 연구가설과 연결하여 다음과 같이 해석할 수 있다.

RQ1(충돌 유형의 형식화 및 탐지 가능성)에 대응하는 가설 H1은 지지된다. 충돌 탐지기를 적용한 모든 그룹에서 메모리 일관성 유지율 100%를 기록한 반면, 미적용 Baseline은 0%를 기록하였다. 이는 충돌 탐지 모듈이 메모리 일관성 확보의 필요조건임을 실증적으로 보여준다.

RQ2(계층형 일관성 모델의 유효성)에 대응하는 가설 H2는 부분적으로 지지된다. StrongOnly는 Baseline과 동일한 정확도(71.1%)를 보여, 단일 일관성 수준만으로는 충돌

해결이 불가능함을 확인하였다. 반면 계층형 일관성을 적용한 FullProposed 는 93.3%의 높은 정확도를 달성하여, 작업 메모리의 강한 일관성과 장기 메모리의 최종 일관성의 조합이 효과적임을 입증하였다.

RQ3(충돌 해소 전략 간 비교)에 대응하는 가설 H3 은 지지된다. TimestampPriority(91.1%)가 TrustScorePriority(62.2%)와 LeaderMediation(62.2%)보다 현저히 높은 정확도를 보였으며, 이는 타임스탬프 기반 전략이 구조적 충돌 해결에 가장 효과적임을 의미한다. 신뢰도 우선 및 리더 중재 전략은 의미적 충돌에서 26.7%로 하락하여, 단일 규칙 기반 접근만으로는 의미적 충돌을 적절히 해결할 수 없음을 확인하였다.

RQ4(Judge Agent 의 의미적 충돌 탐지 유효성)에 대응하는 가설 H4 는 지지된다. Judge Agent 를 포함한 그룹은 의미적 충돌에서 80.0%의 정확도를 기록하여, Judge Agent 가 없는 그룹들 대비 높은 성능을 보였다. 이는 LLM 기반 의미 판단이 구조적 방법으로 탐지 불가능한 의미적 충돌 해결에 효과적임을 입증한다.

RQ5(소거 실험을 통한 모듈별 기여도 확인)에 대응하는 가설 H5 는 지지된다. Judge Agent 제거 시 의미적 충돌 정확도가 하락하고, 사전 필터링 제거 시 LLM 호출이 3 배 증가하고 구조적 충돌 정확도가 53.3%p 하락하여, 두 모듈의 상호 보완적 기여가 정량적으로 입증되었다.

기대와 다른 결과에 대한 해석으로서, 다음과 같이 두 가지 사항을 별도로 논의한다.

StrongOnly(Group 2)의 낮은 성능에 대한 해석: StrongOnly 는 Baseline 과 동일한 종합 정확도(71.1%)와 메모리 일관성 유지율 0%를 기록하였다. 이는 직관적으로 "강한 일관성을 적용하면 충돌이 자동으로 해결될 것"이라는 기대와 상충되는 결과이므로 별도의 해석이 필요하다. 본 연구에서 강한 일관성(Strong Consistency)은 잠금(lock) 기반의 직렬화 메커니즘으로 구현되었으며, 그 역할은 동시 읽기/쓰기 연산의 순서를 보장하는 데 있다. 즉, 강한 일관성은 에이전트가 항상 최신 값을 읽을 수 있도록 보장하지만, 기록된 값이 올바른지—즉 충돌하는 두 값 중 어느 것이 더 적절한지—를 판별하거나 선택하는 기능은 포함하지 않는다. 따라서 두 에이전트가 서로 다른 값을 쓰려 할 때, 강한 일관성은 단지 그 쓰기 순서를 직렬화할 뿐이며, 최종적으로 어떤 값이 메모리에 남을지는 여전히 마지막에 쓴 에이전트의 값(last-write-

wins)에 의해 결정된다. 이로 인해 충돌 탐지기(ConflictDetector)가 없는 StrongOnly 는 Baseline 과 동일하게 충돌을 인식하지 못하고 메모리 불일치를 그대로 허용하게 된다. 이는 일관성 수준(consistency level)과 충돌 해소 메커니즘(conflict resolution mechanism)이 설계상 독립적인 두 차원임을 시사한다. 일관성 수준은 연산의 가시성과 순서를 제어하고, 충돌 해소 메커니즘은 경합하는 값들 사이에서 올바른 값을 선택하는 역할을 담당한다. 두 요소가 함께 설계될 때 비로소 실질적인 메모리 신뢰성이 확보된다는 점이 본 실험에서 실증적으로 확인되었다.

Ablation2 의 동시 쓰기 충돌 정확도가 낮은 이유: Ablation2 의 동시 쓰기 충돌 정확도가 낮은 이유는 Judge Agent 가 버전 문자열과 같은 구조적 정보의 의미를 적절히 해석하지 못하기 때문으로, LLM 기반 Judge 가 자연어 의미 충돌에 효과적이거나 구조적 판별에는 한계가 있음을 의미한다.

이상의 결과를 종합하면, 제안 방식(FullProposed)은 전체 시나리오 평균 93.3%의 정확도와 100%의 메모리 일관성 유지율을 달성하여, 기존의 단순한 last-write-wins 방식이나 단일 중재 전략 방식 대비 명확한 개선 효과를 입증하였다. 본 연구의 핵심 기여는 (1) 멀티 에이전트 공유 메모리 충돌 유형의 형식적 정의, (2) 계층형 일관성 모델과 구조적 사전 필터링 및 Judge Agent 의 통합 설계, (3) 8 개 비교군과 소거 실험을 통한 각 모듈의 개별 기여도에 대한 정량적 입증이다.

실용적 관점에서, 멀티 에이전트 시스템의 공유 메모리에는 최소한의 충돌 탐지 및 버전 관리 메커니즘이 필수적이며, 타임스탬프 기반 구조적 필터링은 구현이 간단하면서도 구조적 충돌 해결에 높은 정확도를 제공한다. LLM 기반 Judge Agent 는 의미적 충돌에 효과적이거나, 비용과 지연 시간 변동성을 고려하여 사전 필터링과의 통합이 필수적이다. 다만 본 결과는 3 개 에이전트, 통제된 실험 환경, DeepSeek-chat 단일 모델, 15 개 의미적 충돌 쌍이라는 제한된 조건에서 도출되었으므로, 에이전트 수 확장, 다양한 LLM 모델 비교, 실제 멀티 에이전트 프레임워크와의 통합 실험 등 추가 검증이 필요하다.

결론적으로, 본 실험 결과는 멀티 에이전트 공유 메모리 환경에서 계층형 일관성 모델과 Judge Agent 기반 통합 충돌 해소 메커니즘이 실용적이고 효과적인 설계 방향임을 1 차적으로

입증하며, 향후 LLM 기반 멀티 에이전트 시스템의 신뢰성 향상에 기여할 수 있는 기반 연구로서 의의를 가진다.

제 6 장 결론 및 시사점

이상의 결과를 종합하면, 본 연구의 제안 방식은 멀티 에이전트 공유 메모리 환경에서 의미 있는 가능성을 보였으며, 본 장에서는 이를 바탕으로 결론과 시사점을 제시한다.

6.1 연구 요약

본 연구는 멀티 에이전트 공유 메모리 시스템에서 여러 에이전트가 동일한 메모리 항목에 동시에 접근하고 갱신할 때 발생하는 충돌 문제를 해결하기 위해, 작업 메모리에는 강한 일관성을, 장기 메모리에는 최종 일관성을 적용하는 계층형 일관성 모델을 설계하고, 타임스탬프 우선·신뢰도 우선·리더 에이전트 중재·LLM 기반 Judge Agent 를 통합한 충돌 해소 메커니즘을 제안하는 데 목적을 두었다. 이를 위해 가시성 충돌, 순서성 충돌, 의미적 충돌의 세 가지 충돌 유형을 형식적으로 정의하고, Python 기반 프로토타입과 SQLite 공유 메모리 저장소를 구현하였으며, Planner·Executor·Reviewer 의 세 역할 에이전트가 참여하는 실험 환경에서 8 개 비교군을 대상으로 동시 쓰기 충돌·버전 순서 충돌·의미적 충돌의 세 가지 시나리오를 각 30 회 반복 수행하여 총 720 회의 실험을 실행하였다.

실험 결과, 제안 방식(FullProposed)은 전체 시나리오 평균 93.3%의 충돌 해결 정확도를 기록하여 Baseline(71.1%) 대비 22.2%p 향상된 성능을 보였으며, 충돌 탐지 및 해소 메커니즘을 적용한 모든 비교군에서 메모리 일관성 유지율 100%를 달성하였다. 의미적 충돌 시나리오에서는 Judge Agent 를 포함한 그룹이 80.0%의 정확도를 기록하여, Judge Agent 가 없는 그룹들 대비 높은 성능을 보임으로써 LLM 기반 의미 판단의 유효성을 입증하였다. 또한 소거 실험을 통해 구조적 사전 필터링과 Judge Agent 의 상호 보완적 통합이 정확도와 효율성의 균형을 확보하는 핵심 설계 요소임을 확인하였으며, 제안 방식은 평균 478.7ms 의 지연시간으로 Ablation2(1485.8ms) 대비 3.1 배 낮은 지연시간을 기록하였다.

이러한 결과는 멀티 에이전트 공유 메모리 환경에서 단순한 last-write-wins 방식이나 단일 중재 전략만으로는 다양한 유형의 충돌에 효과적으로 대응하기 어려우며, 계층형 일관성 모델과 구조적-의미적 통합 충돌 해소 메커니즘이 실용적이고 효과적인 설계 방향임을 시사한다. 특히 본 연구는 메모리 구조, 일관성 수준, 충돌 중재 전략, 의미적 조정을 하나의 프레임워크로 통합하고, 8 개 비교군 및 소거 실험을 통해 각 구성 요소의 개별 기여도를 정량적으로 검증함으로써, 멀티 에이전트 공유 메모리 시스템의 신뢰성 향상을 위한 구현 가능한 설계 기준을 제시하였다는 점에서 의의를 가진다.

6.2 연구의 시사점

6.2.1 학술적 시사점

첫째, 본 연구는 멀티 에이전트 공유 메모리 환경에서 발생하는 충돌을 가시성 충돌, 순서성 충돌, 의미적 충돌의 세 가지 유형으로 형식적으로 정의하고, 각 유형에 대한 탐지 조건과 해소 기준을 구체화하였다. 기존 연구들이 충돌 문제를 개념적으로만 언급한 것과 달리, 본 연구는 세 충돌 유형을 연산 수준에서 정의하고 실험을 통해 각 유형의 발생 조건과 해소 가능성을 검증함으로써, 멀티 에이전트 메모리 충돌 연구의 개념적 기반을 보다 엄밀하게 정립하였다.

둘째, 본 연구는 분산 시스템 분야의 전통적 일관성 모델을 LLM 기반 멀티 에이전트 메모리 환경에 적용하기 위한 계층형 접근법의 유효성을 실증적으로 검증하였다. 단일 일관성 수준을 전체 메모리에 일률적으로 적용하는 방식이 Baseline 대비 유의미한 개선을 보이지 못한 반면, 작업 메모리와 장기 메모리에 차등 적용하는 계층형 설계가 정확도와 효율성의 균형을 확보할 수 있음을 실험적으로 보임으로써, 일관성 모델 연구에 메모리 계층별 차별화라는 새로운 설계 차원을 추가하였다.

셋째, 본 연구는 구조적 충돌 해소 전략과 의미적 충돌 해소 전략을 동일한 실험 조건에서 비교 평가할 수 있는 체계적 실험 프레임워크를 제시하였다. 특히 소거 실험을 포함한 8 개 비교군 설계를 통해 각 전략의 개별 기여도를 정량적으로 분리·측정할 수 있음을 보임으로써, 향후 유사한 멀티 에이전트 메모리 연구의 평가 방법론에 참조 가능한 실험 설계 모형을 제공한다.

6.2.2 실무적 시사점

첫째, 실제 멀티 에이전트 시스템을 구축할 때 공유 메모리에는 최소한의 충돌 탐지 및 버전 관리 메커니즘이 필수적으로 포함되어야 한다. Baseline 의 메모리 일관성 유지율이 0%였던 결과는, 충돌 관리 기능이 없는 공유 메모리는 멀티 에이전트 협업 환경에서 사실상 일관된 상태를 유지할 수 없음을 의미한다.

둘째, 타임스탬프 기반 구조적 사전 필터링은 구현이 비교적 간단하면서도 구조적 충돌에서 높은 해결 정확도를 제공하므로, 실무적으로 우선 도입할 수 있는 경량화된 충돌 해소 기법으로 고려될 수 있다.

셋째, LLM 기반 Judge Agent 는 의미적 충돌 해결에 효과적이지만, API 호출에 따른 지연시간 증가와 변동성이라는 실무적 비용을 수반한다. 따라서 실무적으로는 구조적 필터링으로 1 차 선별한 후, 자연어 의미 충돌이 의심되는 경우에 한해 선별적으로 Judge Agent 를 호출하는 2 단계 아키텍처가 비용 대비 효과적인 설계 전략으로 권장된다.

넷째, 본 연구의 계층형 일관성 설계는 AutoGPT [13], MetaGPT [14], CrewAI [15] 등 최근 주목받는 멀티 에이전트 프레임워크의 공유 메모리 모듈 설계에도 직접적으로 참조될 수 있는 실무적 함의를 지닌다.

6.3 연구의 한계

본 연구는 다음과 같은 한계를 가지며, 이러한 한계는 연구 결과의 해석과 일반화에 일정한 제한을 부여한다.

첫째, 본 연구의 실험은 3 개 에이전트(Planner, Executor, Reviewer)라는 제한된 규모와 통제된 실험 환경에서 수행되었다. 에이전트 수가 증가하면 동시 접근 빈도와 충돌 발생 패턴이 달라질 수 있으며, 특히 리더 에이전트 기반 중재 전략은 에이전트 규모에 따라 성능 특성이 변화할 가능성이 있다. 따라서 본 연구에서 확인된 각 비교군의 정확도와 지연시간 수치는 더 많은 에이전트가 참여하는 실제 멀티 에이전트 협업 환경에서 동일하게 재현된다고 일반화하는 데에는 신중함이 요구된다.

둘째, 의미적 충돌 시나리오는 15 개의 사전 정의된 모순 쌍을 기반으로 평가되었으며, 정답 레이블(Ground Truth)은 연구자의 단독 판단에 따라 설정되었다. Judge Agent 의 80.0% 정확도는 이러한 제한된 시나리오 내에서의 성능으로 이해되어야 하며, 보다 다양하고 실제적인 의미 충돌 상황에서의 일반화 가능성은 추가 검증이 필요하다.

셋째, 본 연구는 DeepSeek-chat [10] 단일 LLM 모델만을 Judge Agent 로 사용하였기 때문에, Judge Agent 의 성능이 사용된 특정 모델의 특성에 의존할 가능성이 있다. 다른 LLM 모델을 Judge Agent 로 사용할 경우 의미적 충돌 탐지 정확도와 지연시간이 달라질 수 있으며, LLM API 응답 시간의 자연적 변동성이 실험 결과의 지연시간 분포에 영향을 미쳤다.

넷째, 본 연구에서 설계된 프로토타입은 SQLite 기반의 단일 머신 공유 메모리 구조를 사용하였으며, 네트워크로 분리된 분산 환경에서의 성능은 검증하지 않았다. 실제 멀티 에이전트 시스템이 분산 네트워크 환경에서 운영될 경우, 네트워크 지연, 메시지 손실, 노드 장애, 네트워크 분할과 같은 추가적인 변수가 메모리 일관성과 충돌 해소 메커니즘의 동작에 영향을 미칠 수 있다.

6.4 향후 연구 방향

이상의 한계를 바탕으로, 향후 연구는 다음과 같은 방향으로 확장될 필요가 있다.

1. 에이전트 수를 10 개, 20 개, 50 개 등으로 점진적으로 확장하고, 역할의 종류를 다양화하여 에이전트 규모와 역할 다양성이 충돌 해소 메커니즘의 성능에 미치는 영향을 체계적으로 분석한다.

2. 보다 다양하고 실제적인 의미적 충돌 데이터셋을 구축하고, 복수의 평가자가 참여하는 정답 레이블링 체계를 도입하며, GPT-4, Claude, Gemini 등 다양한 LLM 모델을 Judge Agent 로 비교 실험한다.

3. 본 연구의 프로토타입을 분산 환경으로 확장하여 네트워크 지연, 메시지 손실, 노드 장애, 네트워크 분할 등이 계층형 일관성 모델과 충돌 해소 메커니즘에 미치는 영향을 검증하고, CAP 정리 [6] 관점에서 일관성-가용성-분할 내성 간 트레이드오프 특성을 분석한다.

4. 제안한 계층형 일관성 모델과 충돌 해소 메커니즘을 AutoGPT [13], MetaGPT [14], CrewAI [15] 등 실제 멀티 에이전트 프레임워크의 공유 메모리 모듈에 통합하여, 코드 생성, 데이터 분석, 문서 작성과 같은 구체적인 응용 과업에서 실증적 유효성을 평가한다.

5. 충돌 유형의 사전 확률, 에이전트 신뢰도 변화 추이, 시스템 부하 상태 등을 종합적으로 고려하여 Judge Agent 호출 여부와 전략 선택을 실시간으로 결정하는 강화학습 기반의 적응형 충돌 해소 메커니즘을 연구한다.

참고문헌

- [1] Gu, Y.; et al. (2024). Memory in Multi-Agent Systems: Shared Memory and Distributed Memory Paradigms. arXiv preprint.
- [2] Mao, Y.; et al. (2024). AMA: Adaptive Memory Management for Multi-Agent Systems with Role-Based Agents. Proceedings of NeurIPS.
- [3] Herlihy, M. P.; & Wing, J. M. (1990). Linearizability: A Correctness Condition for Concurrent Objects. ACM Transactions on Programming Languages and Systems, 12(3), 463-492. <https://doi.org/10.1145/78969.78972>
- [4] Vogels, W. (2009). Eventually Consistent. Communications of the ACM, 52(1), 40-44. <https://doi.org/10.1145/1435417.1435432>
- [5] Ahamad, M.; Neiger, G.; Burns, J. E.; Kohli, P.; & Hutto, P. W. (1995). Causal Memory: Definitions, Implementation, and Programming. Distributed Computing, 9(1), 37-49. <https://doi.org/10.1007/BF01784241>
- [6] Brewer, E. A. (2000). Towards Robust Distributed Systems. Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC). <http://www.podc.org/podc2000/brewer.html>
- [7] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>
- [8] Garcia-Molina, H.; & Salem, K. (1987). Sagas. Proceedings of the ACM SIGMOD International Conference on Management of Data, 249-259. <https://doi.org/10.1145/38713.38742>
- [9] Terry, D. B.; Theimer, M. M.; Petersen, K.; Demers, A. J.; Spreitzer, M. J.; & Hauser, C. H. (1995). Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System. Proceedings of the ACM Symposium on Operating Systems Principles (SOSP), 172-183. <https://doi.org/10.1145/224056.224070>
- [10] DeepSeek. (2024). DeepSeek API Documentation. <https://api.deepseek.com>
- [11] OpenAI. (2024). OpenAI Python SDK Documentation. <https://github.com/openai/openai-python>
- [12] SQLite. (2024). Write-Ahead Logging. <https://www.sqlite.org/wal.html>
- [13] AutoGPT Team. (2024). AutoGPT: Autonomous AI Agent Framework. <https://github.com/Significant-Gravitas/AutoGPT>
- [14] MetaGPT Team. (2024). MetaGPT: Multi-Agent Meta Programming Framework. <https://github.com/geekan/MetaGPT>

- [15] CrewAI Team. (2024). CrewAI: Framework for Orchestrating Role-Playing AI Agents.
<https://github.com/crewAIInc/crewAI>

국문 초록

멀티 에이전트 공유 메모리 시스템에서의

계층형 일관성 모델 및

충돌 해소 메커니즘 설계

위바오치

Major of Computer Science and Technology

Graduate School of Youngsan University

담당교수 : 백 건 효

멀티 에이전트 시스템에서 공유 메모리는 에이전트 간 상태 동기화와 지식 공유를 위한 핵심 구성 요소이다. 그러나 여러 에이전트가 동시에 동일한 메모리 항목에 접근하고 갱신할 때 가시성 충돌, 순서성 충돌, 의미적 충돌이 발생할 수 있으며, 이를 적절히 관리하지 못하면 메모리 상태의 불일치가 누적되어 시스템의 신뢰성을 크게 훼손할 수 있다. 본 연구는 이러한 문제를 해결하기 위해 작업 메모리에는 강한 일관성을, 장기 메모리에는 최종 일관성을 적용하는 계층형 일관성 모델을 제안하고, 타임스탬프 우선·신뢰도 우선·리더 에이전트 중재·LLM 기반 Judge Agent 를 통합한 충돌 해소 메커니즘을 설계하였다. 가시성 충돌, 순서성 충돌, 의미적 충돌의 세 가지 충돌 유형을 형식적으로 정의하고, Python 기반 프로토타입과 SQLite 공유 메모리 저장소를 구현하여 Planner·Executor·Reviewer 의 세 역할 에이전트가 참여하는 실험 환경에서 8 개 비교군을 대상으로 3 종의 충돌 시나리오를 각 30 회 반복 수행하는 실험을 실행하였다. 실험 결과, 제안 방식(FullProposed)은 전체 시나리오 평균 93.3%의 충돌 해결 정확도를 기록하여 Baseline(71.1%) 대비 22.2%p 향상된 성능을 보였으며,

충돌 탐지 및 해소 메커니즘을 적용한 모든 비교군에서 메모리 일관성 유지율 100%를 달성하였다. 의미적 충돌 시나리오에서는 Judge Agent 를 포함한 그룹이 80.0%의 정확도를 기록하여 LLM 기반 의미 판단의 유효성을 입증하였으며, 소거 실험(Ablation Study)을 통해 구조적 사전 필터링과 Judge Agent 의 상호 보완적 통합이 정확도와 효율성의 균형을 확보하는 핵심 설계 요소임을 확인하였다.

키워드: 멀티 에이전트 시스템, 공유 메모리, 계층형 일관성 모델, 충돌 해소 메커니즘, Judge Agent, LLM

ABSTRACT

Design of a Hierarchical Consistency Model and Conflict Resolution Mechanism in Multi-Agent Shared Memory Systems

YU BAOQI

Major of Computer Science and Technology

Graduate School of Youngsan University

Advisor: 백 건 효

In multi-agent systems, shared memory serves as a core component for state synchronization and knowledge sharing among agents. However, when multiple agents concurrently access and update the same memory entries, visibility conflicts, ordering conflicts, and semantic conflicts can arise. If not properly managed, accumulated memory inconsistencies can severely undermine system reliability. To address these challenges, this study proposes a hierarchical consistency model that applies strong consistency to working memory and eventual consistency to long-term memory, and designs an integrated conflict resolution mechanism combining timestamp priority, trust score priority, leader agent mediation, and an LLM-based Judge Agent. Three conflict types—visibility, ordering, and semantic—are formally defined. A Python-based prototype with SQLite shared memory storage was implemented, and experiments were conducted with three role-based agents (Planner, Executor, Reviewer) across 8 comparison groups under 3 conflict scenarios with 30 iterations each, totaling 720 experimental runs. Experimental results demonstrate that the proposed approach (FullProposed) achieved an average conflict resolution accuracy of 93.3% across all scenarios, representing a 22.2%p improvement over Baseline (71.1%). All groups equipped with conflict detection and resolution mechanisms achieved 100% memory consistency rates. In semantic

conflict scenarios, groups incorporating the Judge Agent achieved 80.0% accuracy, validating the effectiveness of LLM-based semantic judgment. Ablation studies confirmed that the complementary integration of structural pre-filtering and the Judge Agent is a key design factor for balancing accuracy and efficiency. The proposed approach recorded an average latency of 478.7ms, approximately $3.1\times$ lower than the configuration without pre-filtering (1485.8ms), demonstrating that pre-filtering effectively controls both the mean and variance of latency while maintaining high accuracy.

Keywords: Multi-Agent System, Shared Memory, Hierarchical Consistency Model, Conflict Resolution Mechanism, Judge Agent, LLM

APPENDIX

실험 코드: <https://github.com/ybq9430/multi-agent-shared-memory-experiment>